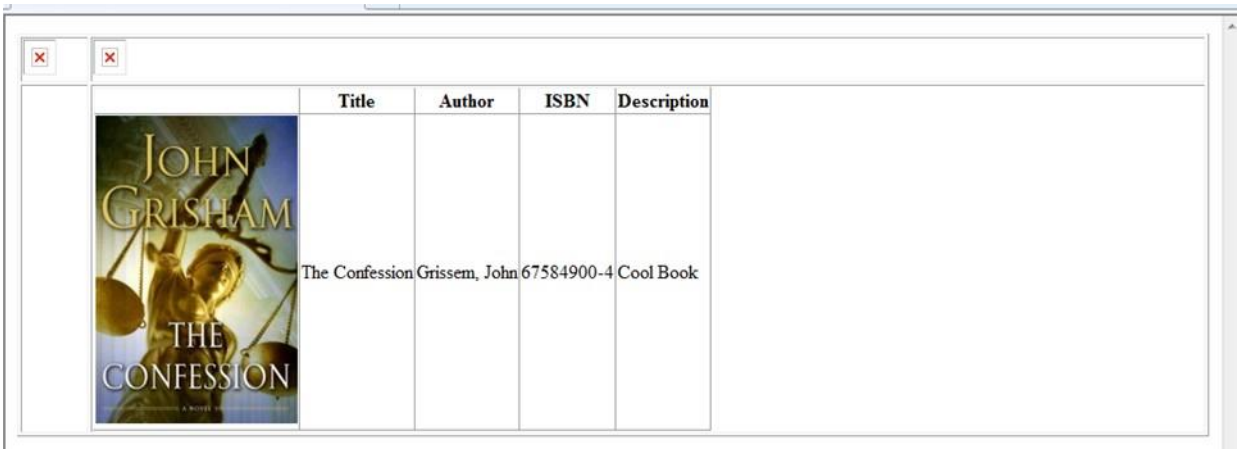
ITWP 2300

BUILDING DYNAMIC, INTELLIGENT WEB BASED SOLUTIONS WITH ASP.NET



PROJECT 2

In this unit we will begin the development of the customer interface, our doorway to the world. In support of this, we will be creating menus based on the dynamic data available in the database.

Welcome to my Book World!

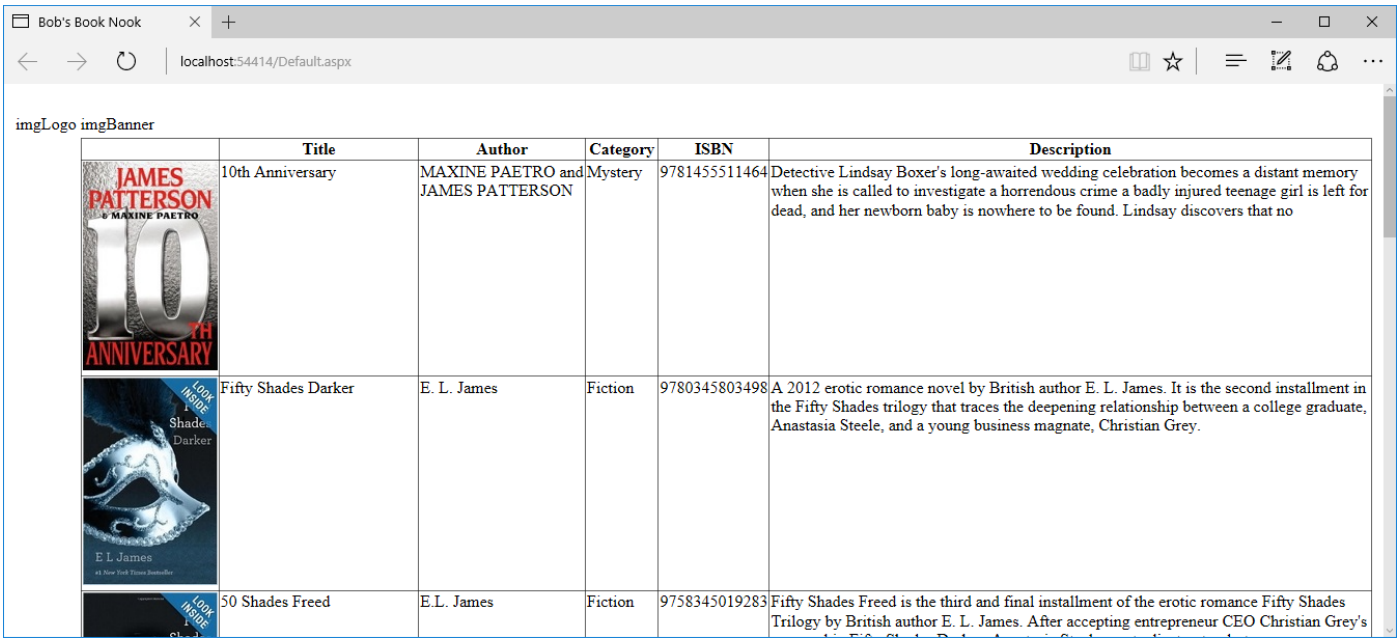
Before you begin, copy bobsbooknook1 folder to a folder named bobsbooknook2.
Open bobsbooknook2 in Visual Studio and make sure the project works.

Ed Note: In this, as well as any Visual Studio project, page names can be capitalized or not. Just make sure all page references match the coded capitalization.

- Insert a new blank web form named Default.aspx.
- Set the title tag in Default.aspx to Bob's Book Nook.
- Add lblErrorMessage to the top of the design view, Default.aspx.
- Working in design view of Default.aspx, insert a 2x2 table with border set to 1.
    - In row 1 column 1 insert the text "imgLogo". In row 1 column 2 insert the text "ImgBanner"
    - In row 2 column 2 insert a grid view object from the toolbox naming it gvCatalog.
- Using the HTML Code version of gvDIsplay from the Admin page, modify gvCatalog so that it matches in functionality.
- Double check the name of the object and reset it to gvCatalog if necessary.
- Add the **td** CSS script from Admin.aspx to Default.aspx.
- In Default.aspx.cs comment **out using System.Linq;**
- Add global variable `string ownerID; to Default.aspx.cs.`
- Copy FormatImageUrl from Admin.aspx.cs and place it in Default.aspx.cs outside of Page_Load.
- Insert the following try catch block in Page_Load.

```
string uploadPath = Server.MapPath("~/App_Data/");
        try
        {
            ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
            ownerID = ownerID.Trim();
            Response.Write(ownerID + "<br />"); //error checking only.
        }
        catch { }
```

- Copy the code from Admin.btnViewInventory_click(… paste it in Page_Load just under the try catch block.
- Make the following change to use the global variable instead of a non-existent textbox :
- `string sqlCommand = "SELECT Title, ISBN, Author, Category, Image, Description FROM " + ownerID + "BOOKS";`
- Edit the SQL statement to use gvCatalog.
- `myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);`
- Execute your program.



Though the images may be different, your web page should look similar to the image above.

If you have issues, the code for Default.aspx and the code for Default.aspx.cs are shown below.

Default.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs" Inherits="_Default" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Bob's Book Nook</title>
       <style type="text/css">
              .auto-style1 {
                     width: 100%;
                     border-style: solid;
                     border-width: 1px;
              }
              td {
       vertical-align: top ;
   }
       </style>
</head>
<body>
    <form id="form1" runat="server">
        <div>
              <asp:Label ID="lblErrorMessage" runat="server"></asp:Label>
                     <br />
                     <table class="auto-style1">
                            <tr>
                                   <td>imgLogo</td>
                                   <td>imgBanner</td>
                            </tr>
                            <tr>
                                   <td> </td>
                                   <td>
                                          <asp:GridView ID="gvCatalog" runat="server"
AutoGenerateColumns="False">
                                          <Columns>
```

```
                                        <asp:TemplateField>
                                         <ItemTemplate>
                                       <asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string)
FormatImageUrl( (string) Eval("Image")) %>' />
                                         </ItemTemplate>
                                        </asp:TemplateField>
                                        <asp:BoundField DataField="Title" HeaderText="Title" />
                                        <asp:BoundField DataField="Author" HeaderText="Author" />
                                        <asp:BoundField DataField="Category" HeaderText="Category"
/>
                                        <asp:BoundField DataField="ISBN" HeaderText="ISBN" />
                                        <asp:BoundField DataField="Description"
HeaderText="Description" />
                                       </Columns>
                                       </asp:GridView>
                                 </td>
                            </tr>
                      </table>
        </div>
    </form>
</body>
</html>
```

Default.aspx.cs

```
using System;
using System.Collections.Generic;
//using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class _Default : System.Web.UI.Page
{
string ownerID;

protected void Page_Load(object sender, EventArgs e)
{
       string uploadPath = Server.MapPath("~/App_Data/");
       try
       {
              ownerID = System.IO.File.ReadAllText(uploadPath + "ownerID.txt");
              ownerID = ownerID.Trim();
              Response.Write(ownerID + "<br />"); //error checking only.
       }
       catch { }
       string sqlCommand = "SELECT Title, ISBN, Author, Category, Image, Description FROM " + ownerID
+ "BOOKS";

       myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
}
protected string FormatImageUrl(string url)
{
       if (url != null && url.Length > 0)
              return ("~/" + url);
       else return null;
}
}
```

ITWP 2300 eBook Store Project 2

Reuse Code!

What we have done is reuse (copy) that part of Admin.aspx and Admin.aspx.cs that we have already debugged and made to work, making very few minor adjustments.

We will look at the Response.Redirect issue later in this document.

Everything should work as planned. The real question is what order and which fields do we want to show the customer?

Note: You are responsible for a banner image and an image for the menu.

Loading the Menus.

When the web page loads, the complete inventory of books is available in gvCatalog. We can discuss modifications to this in part 5 when we complete finishing touches on the project.

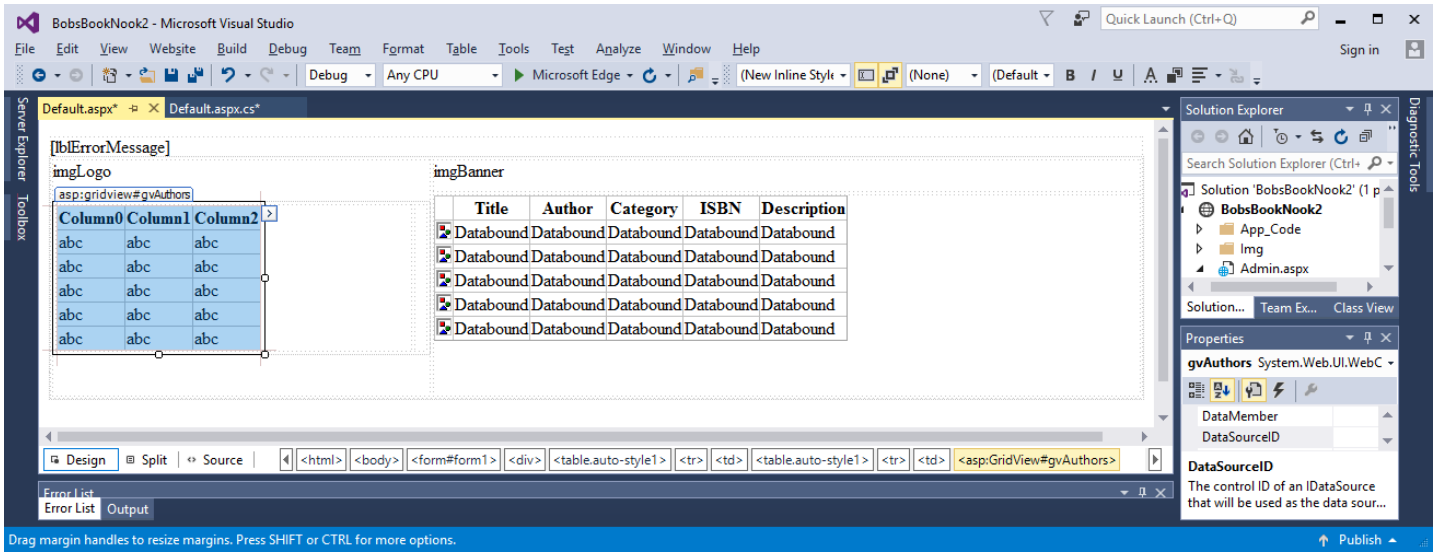
In addition to the catalog, we want to create two dynamic menu lists (we will use gridviews). The first is Authors. We will load a non-duplicated list of authors from the database. If the visitor selects any of the authors, gvCatalog will be loaded with books by that Author.

The second list will contain the categories of each of the books (again non-duplicated) where visitors can list all books in a single category.

[Bestsellers] and [Clear Search] will be added as buttons later in the project.

The addition of the two menu items is a matter of placement of the grid views, editing grid view columns and calls to **myDatabaseConnection.executeSQL**. All menu buttons should be coded to "View" OR "Select", not a mix of both. *Note that we will be adding the Click To Purchase column in gvCatalog near the end of this Project.*

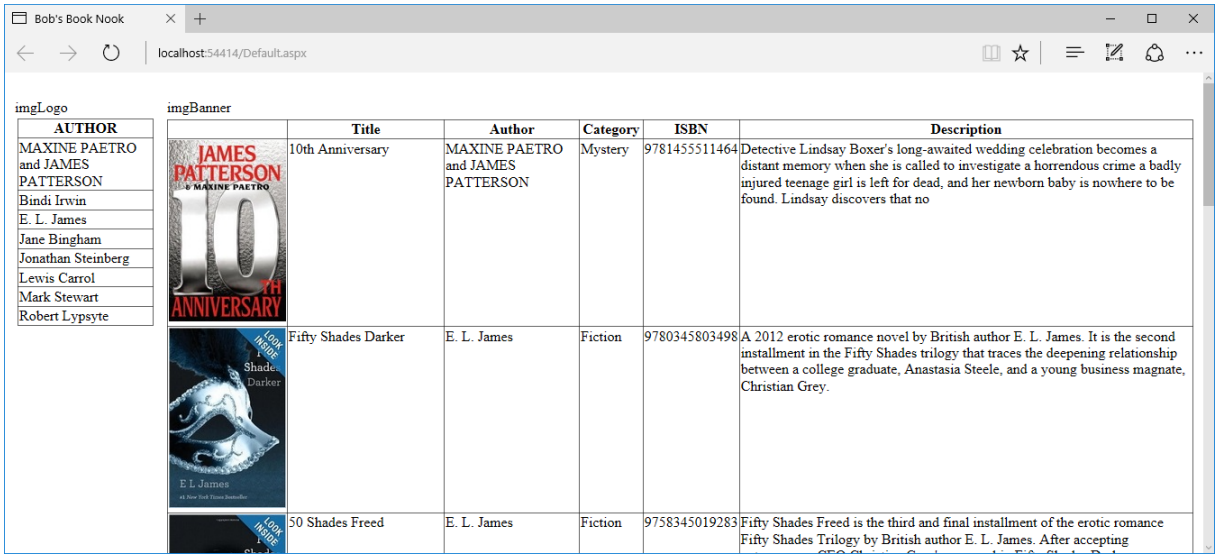
By Author:



Insert a 1 row 2 column table in row 2 column 1 of the current table.
Start by adding gvAuthors in row 1, column 1 of the new table.
Add the following call (bold code) to executeSQL at the bottom of Page_Load in Default.aspx.cs.

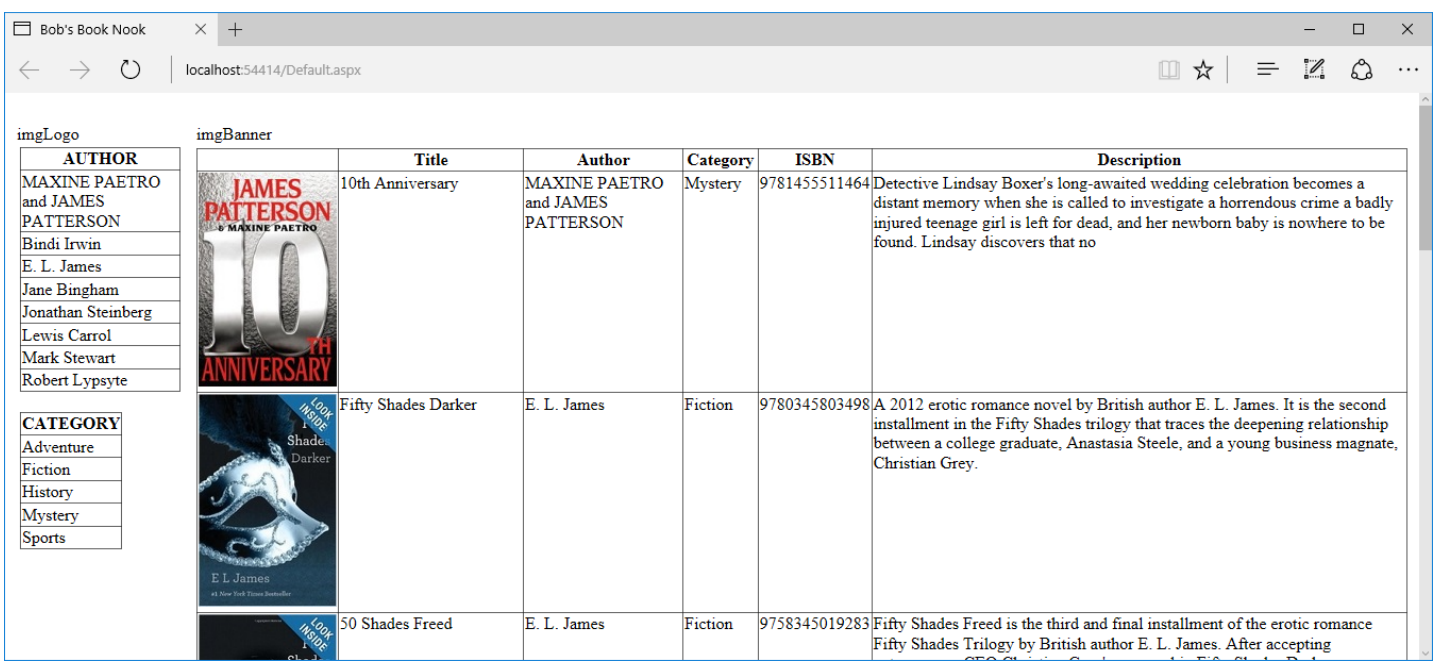
```
string sqlCommand = "SELECT Title, ISBN, Author, Category, Image, Description FROM " + ownerID +
"BOOKS";
myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);

//load author menu
sqlCommand = "SELECT DISTINCT AUTHOR FROM " + ownerID + "BOOKS";
myDatabaseConnection.executeSQL(sqlCommand, ref gvAuthors, ref lblErrorMessage);
```



We see only a single entry for each author. One would say "The list should be sorted." While I would agree, we would have to modify the database to include fname and lname. This would take a change in the XML file from the publisher, and we are too small to risk asking the publisher for much of anything. ☺

Go ahead and add gvCategories under gvAuthors . Using the sql statement for gvAuthors, add an appropriate SQL statement to display category order by category.

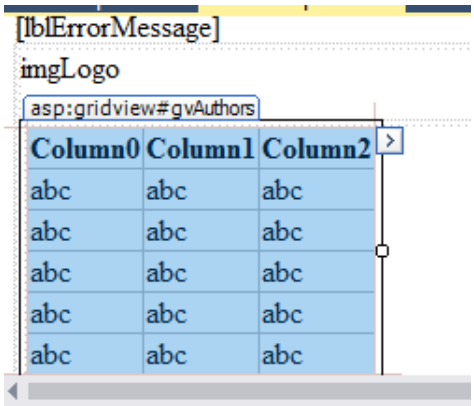


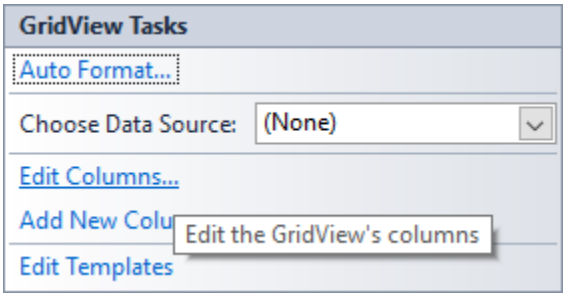
So far so good.

Hint: Copy the Authors statements down and modify for table field Category and gvCategories.

The ability to modify by menu selection.

Making the Grid View Respond Properly:

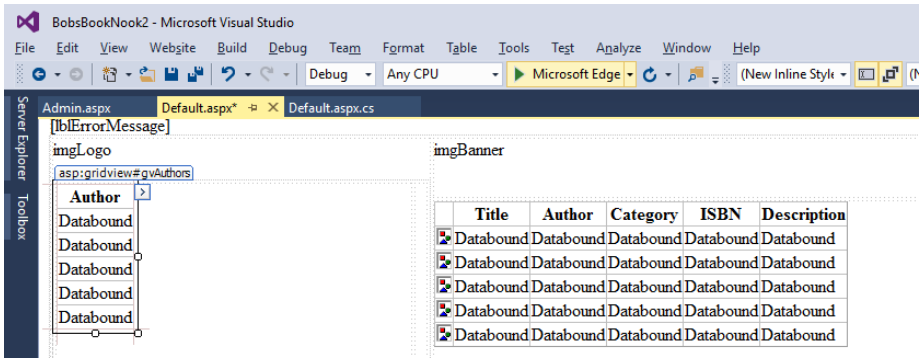




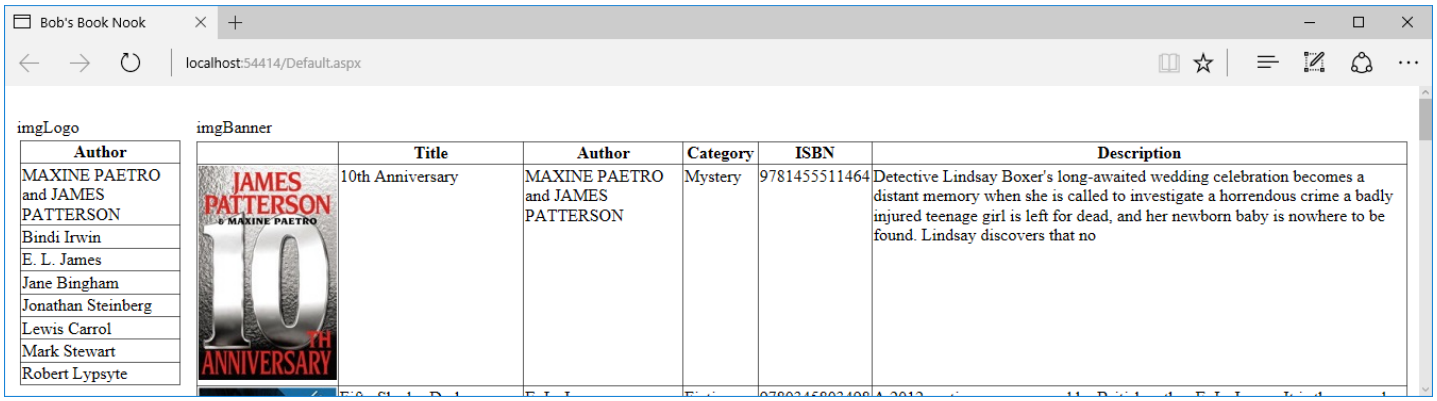
Set up gvAuthors by clicking on the right arrow then edit columns.

Insert a BoundField

- set Header Text to Author
- set DataField to Author
- Uncheck Auto-generated fields
- Execute your app.



Design View



Execution View

When you execute the application at this point, it will not look much different than the plain old gridview we saw in the previous step. The problem is that we need to click on an author and change the display in gvCatalog. Read on!

Reopen gvAuthors for editing.

Click the + next to CommandField.

Choose Select.

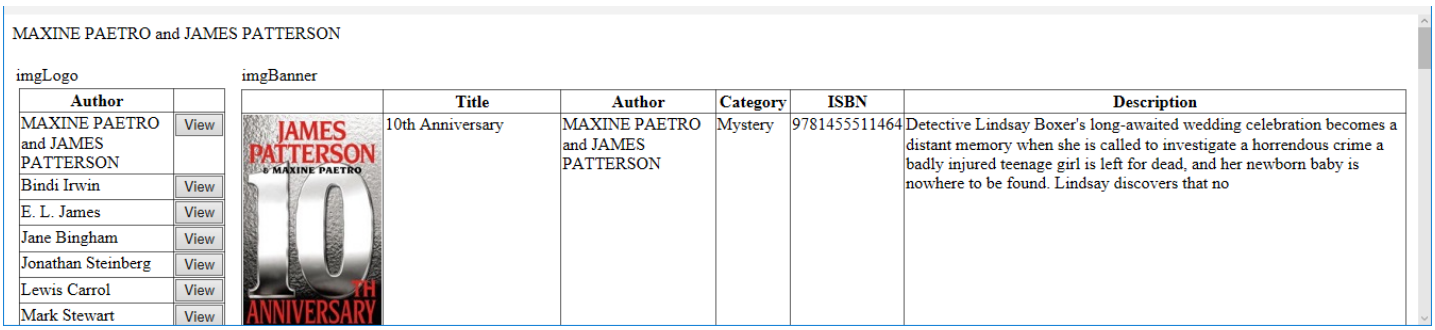
- Set Button Type to **Button**.
- Change SelectText to **View**.
- Set CausesValidation to **False**.
- ShowCancelButton to **False**.
- Verify that ShowSelectButton is set to **True**.

Click [OK] to save the changes.

Back in the web form,

- Set the Property for gvAuthors DataKeyNames to Author.
- Double click the gridview to bring up the selected_index_change method. Insert the following command:

```
protected void gvAuthors_SelectedIndexChanged(…
{
Response.Write(gvAuthors.SelectedValue);
}
```



You should see the value from the first column when you click view.

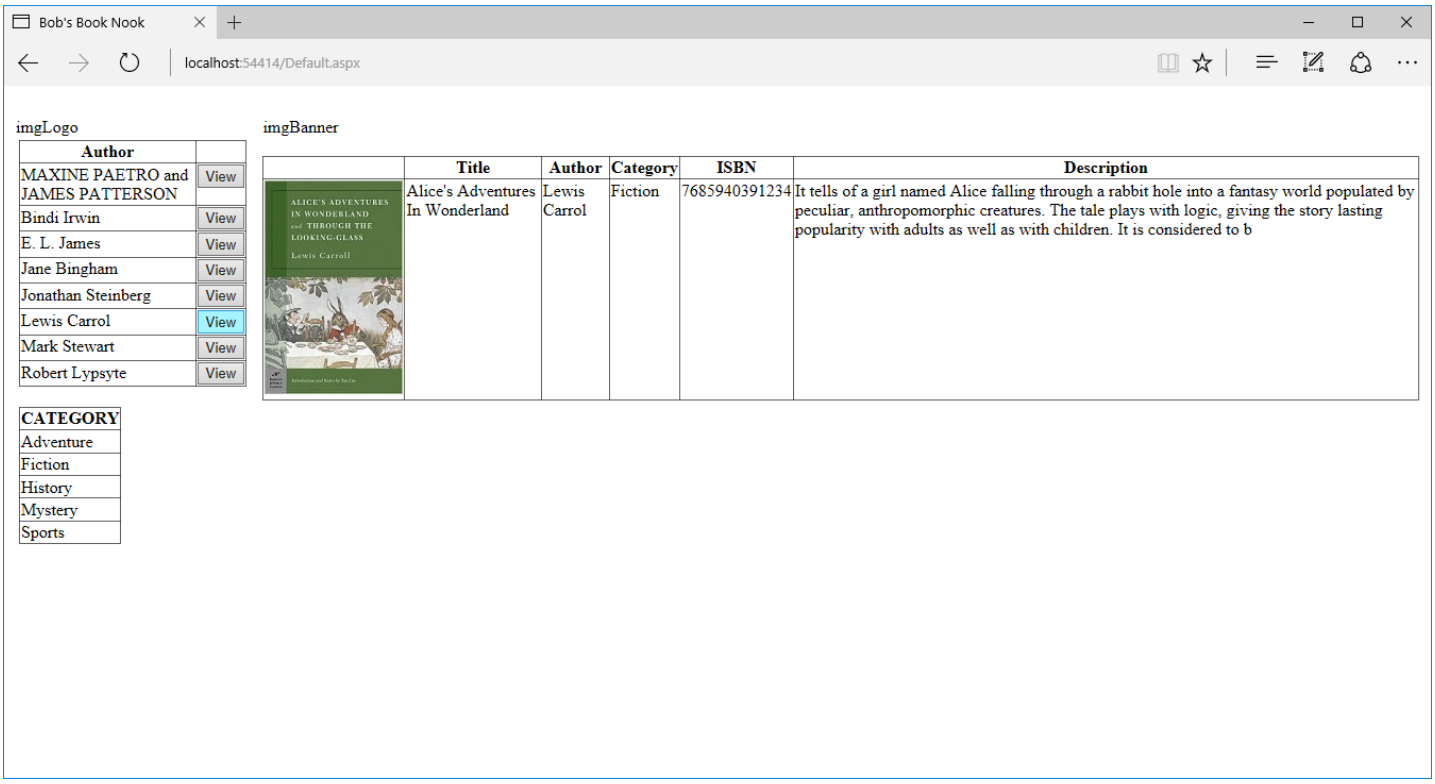
Ed Note: It does not matter if the view button is in front of or after the Author's name.

Make the following change to gvAuthors_SelectedIndexChanged.

```
protected void gvAuthors_SelectedIndexChanged(…
    {
      // Response.Write(gvAuthors.SelectedValue);
        string sqlCommand = "SELECT * FROM " + ownerID + "BOOKS WHERE AUTHOR = '"
+ gvAuthors.SelectedValue + "'";

        myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref
lblErrorMessage);
}
```

Make sure you set the gridview to gvCatalog and not gvAuthors.



As you can see, only author Lewis Carrol shows up in gvCatalog.

By Category

Open gvCategories for editing.

Insert a BoundField

- set Header Text to Category
- set DataField to Category
- Uncheck Auto-generated fields

Click the + next to CommandField.

Choose Select.

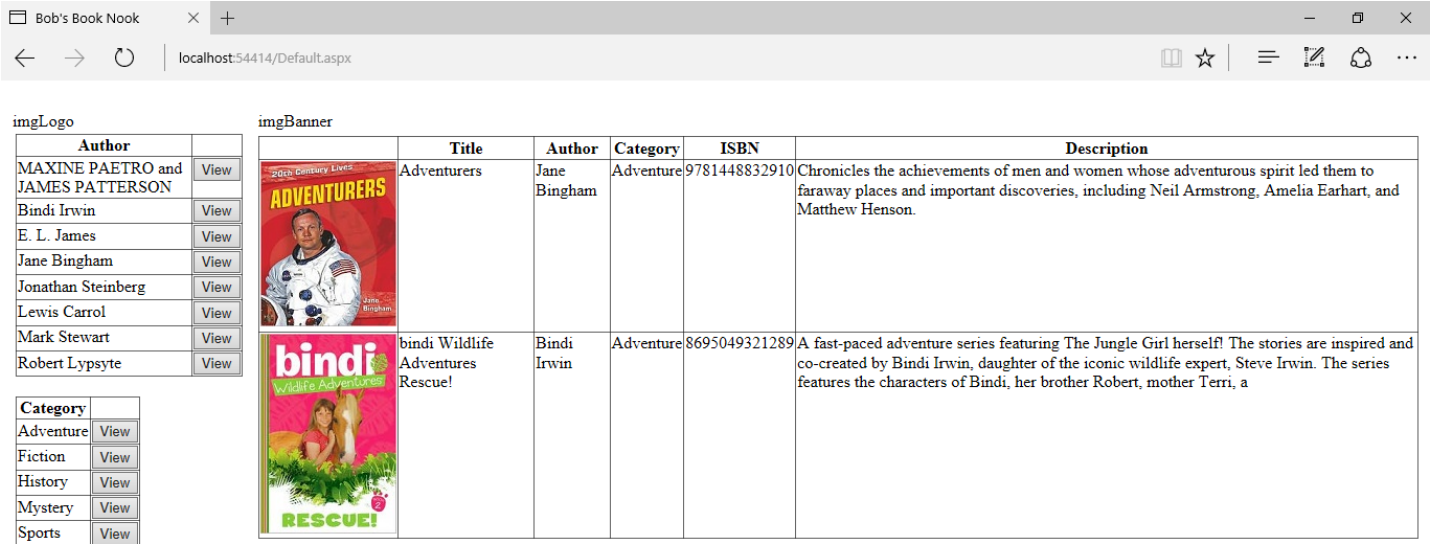
- Set Button Type to Button.
- Change SelectText to View.
- Set CausesValidation to False.
- ShowCancelButton to False.
- Make sure that ShowSelectButton is set to True.

Click [OK] to save the changes.

Back in the web form,

- Set the Property for gvCategories.DataKeyNames to Category.
- Double click the gridview to bring up the selected_index_change method. Insert the following command:

```
protected void gvCategories_SelectedIndexChanged(…
{
string sqlCommand = "SELECT * FROM " + ownerID + "BOOKS WHERE CATEGORY = '" +
gvCategories.SelectedValue + "'";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvCatalog, ref lblErrorMessage);
}
```



You can see in the figure above that selecting Adventure we see the two books on adventure.

Hot Wire gvCatalog

We need to allow the consumer to select a book and place it in the shopping cart. We should make the shopping cart first, as simple as it will be at this point, then connect Default to ShoppingCart.

Add ShoppingCart.aspx to the project.

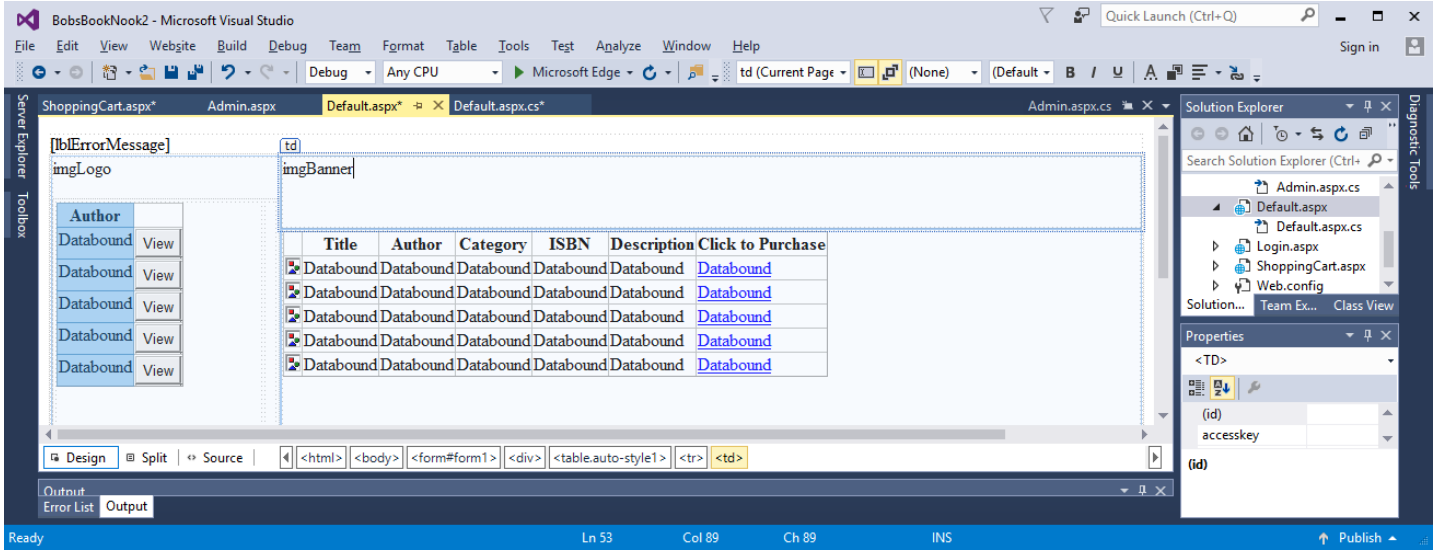
Set the title tag to **Shopping Cart.**

Creating a Connection to The Shopping Cart

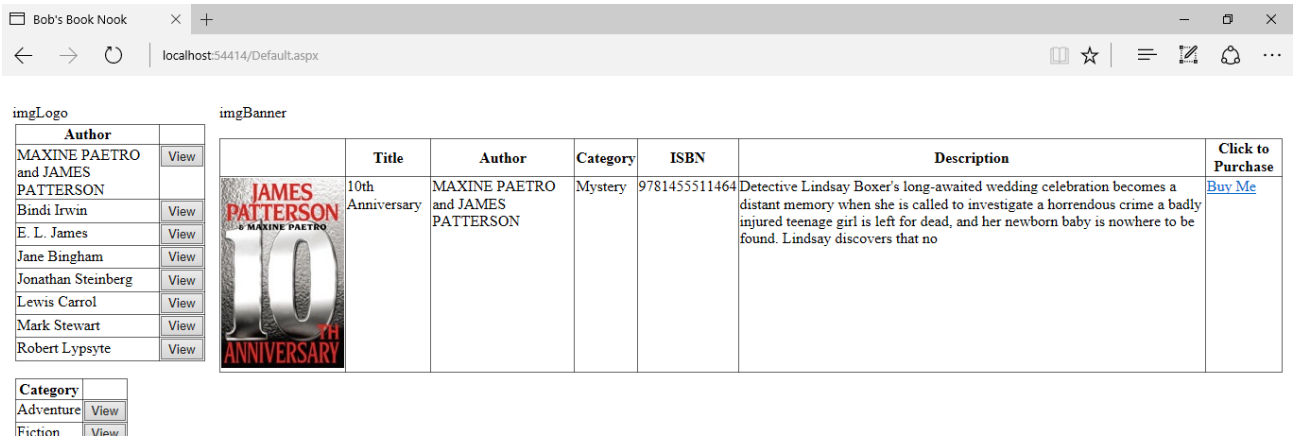
One last modification to gvCatalog:

- Select the **gvCatalog** object, click on the ">" then click on E**dit Columns**.
- Insert a new HyperLinkField
- Set header text to "**Click to Purchase**"
- Set Text to **ISBN**
- Insert Visible is set to **True**
- Navigate URL is set to **~/ShoppingCart.aspx**
- Target is set to **_top**
- Visible is set to **True**.
- DataNavigateUrlFields is set to **ISBN**.
- DataNavigateUrlFormat is set to **ShoppingCart.aspx?Id={0}**
- Data Text Field = **ISBN**
- Data Text Format String = **Buy Me**.
- Click on **[Ok]** to save.

Note: Be careful the 0 inside of {0} is the number zero.



In design mode.



About to click on Buy Me.

It Works! We are in ShoppingCart, even though there is nothing displayed.

- *Ed Note: If the ShoppingCart page does not display, validate the spelling making sure the name of the aspx file is ShoppingCart.aspx, the* Navigate URL entry is **~/ShoppingCart.aspx** (no spaces) and that DataNavigateUrlFormat is set to **ShoppingCart.aspx?Id={0}** ie: ={the number 0}.

Importing Information to ShoppingCart.

We do need a way to 'capture' each individual selection on the ShoppingCart page which we will use to eventually display a grid view. There are many ways to handle this including more cookies, creating a database table for each user, or creating some sort of invisible listbox object that we have to make sure maintains state throughout the session… oh, and places items not sold back into the inventory if a customer abandons their shopping cart. After much discussion with colleagues, it was decided that we would use Session variables.

To manage our session is quite simple. The code below is ShoppingCart.Page_Load.

```
protected void Page_Load(…
{
    try
    {
        //Save session variable
        Session[Session.Count.ToString()] = Request.QueryString["id"];
        //display session variable
        for (int i = 0; i < Session.Count; i++)
        {
            Response.Write("<br />" + Session[i.ToString()].ToString());
        }
    }
    catch { Response.Redirect("~/Default.aspx"); }
}
```

9781455511464
9780345803498
6758493021982

ShoppingCart after 3 selections.

An Explanation:

When researching the Microsoft site for Session information, we usually see something that looks like this:
Ed Note: Do NOT place this code in your project.

```
//save session variables  string
firstName = "Jeff"; string lastName =
"Smith"; string city = "Seattle";
Session["FirstName"] = firstName;
Session["LastName"] = lastName; Session["City"] =
city;
```

and

```
//read session variables
string firstName = (string)(Session["First"]); string
lastName = (string)(Session["Last"]); string city =
(string)(Session["City"]);
```

Not much help when we do not know how many actual session variables we will have or what they are named. Try to think of a way to manage the field names. Do we use ISBN as both the index and the contents? Been there, tried that. Tried to use an array. Tried several other methods before coming up with the solution shown in Page_Load above.

Session.Count provides the number of items just like any list object. When we execute the page for the first time, Session.Count returns 0. We will translate the count value to a string and use that for our index.

When we want to utilize the data, we execute the for loop to traverse through the items in the index, "0", "1", "2", etc. until we print out each of the ISBNs that should be in our shopping cart.

Try your program by clicking Buy Me and after the information shows, click back arrow to return to Default.aspx. Click Buy Me on another item (or the same one) and you should see two ISBNs displayed.

What to do to complete this project:

- Make sure your program works as expected.
- Remove all extra Response.Write commands used for debugging as we built our solution.
- Make sure program is executing as expected.
- Make sure that the 2 menu items are coded and that they work correctly.
- Make sure that you can display an ever-growing number of ISBNs on your Shopping Cart page.
- Place a **[Back to Browsing]** button on **ShoppingCart** that redirects the user back to **Default.aspx**.
  Note: When you do this, a situation is invoked that causes the previous ISBN to be added to the list.
  We will be taking care of this issue in Part 3.
- Add a **[To BookStore]** button at the top of Admin.aspx that redirects the user back to **Default.aspx**.
- *Ed Note: Hint:: Look in the Login Code for the format of Response.Redirect.*
- Change the background and foreground colors of the two new grid view objects to complement your banner image.
- DO NOT be concerned with duplicate numbers in the shopping Cart. We will be creating a listbox to control the data in Part 3.

Post 2 URLs. One

- URL will point to **BobsBookNook2/Default.aspx.**